

Offline Velocity Estimation Using Savitzky–Golay Smoothing and a Kalman Filter with Acceleration Input

Yu-Chieh Lee

February 25, 2026

1. Introduction

1.1. Background

Accurate velocity and acceleration estimation from encoder-derived signals is a fundamental requirement in motion analysis, state estimation, and feedback control. In practice, encoder measurements are often affected by quantization, sensor noise, and numerical differentiation errors. These effects are particularly pronounced at very low velocities, where quantization error represents a larger fraction of the true signal, and during computing velocity and acceleration, where high-frequency noise is amplified.

To mitigate these issues, signal smoothing and model-based estimation techniques are commonly employed. Savitzky–Golay (SG) filtering provides local polynomial smoothing that preserves low-order signal trends while attenuating high-frequency noise. Kalman filtering (KF), on the other hand, offers a principled probabilistic framework for fusing noisy measurements with a dynamic model. When acceleration measurements are available, they can be incorporated either as a state or as an external input to improve transient tracking performance.

This project focuses on offline estimation of one-dimensional motion signals using a combined SG and KF pipeline. Measured position, velocity, and acceleration signals are processed to produce smoothed state estimates, which are then evaluated against provided reference (ground-truth) signals.

1.2. Objectives

The goal of this project is to design and evaluate an offline estimation pipeline for improving velocity estimates derived from noisy encoder data. Specifically, the project:

- Applies Savitzky–Golay smoothing to measured position, velocity, and acceleration signals to reduce high-frequency noise.
- Implements a discrete-time Kalman filter with position and velocity as states, treating measured acceleration as an external input.
- Incorporates empirically estimated, potentially correlated measurement noise statistics into the Kalman filter design.
- Evaluates estimation performance using reference signals and analyzes the relationship between estimation error and velocity magnitude.

2. Data and Problem Formulation

2.1. Dataset and Signals

The input dataset is provided as a CSV file with the first column representing physical time in seconds. The time values are stored as floating-point numbers and increase monotonically.

For each sample index k , the following scalar signals describing motion along a single degree of freedom are extracted:

- $mpos(t_k)$: measured position
- $mvel(t_k)$: measured velocity
- $maccel(t_k)$: measured acceleration
- $rpos(t_k)$: reference (ground-truth) position
- $rvel(t_k)$: reference (ground-truth) velocity
- $raccel(t_k)$: reference (ground-truth) acceleration

The measured signals are treated as noisy observations of the underlying motion, while the reference signals are used exclusively for offline evaluation and are not incorporated into the estimation process.

2.2. Ground Truth and Error Definitions

Estimation performance is evaluated by defining measurement and estimation errors relative to the reference signals. The instantaneous measurement errors at tick t_k are defined as

$$n_x(t_k) = mpos(t_k) - rpos(t_k), \quad (1)$$

$$n_v(t_k) = mvel(t_k) - rvel(t_k), \quad (2)$$

$$n_a(t_k) = maccel(t_k) - raccel(t_k). \quad (3)$$

Similarly, the estimation errors associated with the Kalman filter outputs are defined as

$$e_x(t_k) = \hat{p}(t_k) - rpos(t_k), \quad (4)$$

$$e_v(t_k) = \hat{v}(t_k) - rvel(t_k), \quad (5)$$

where $\hat{p}(t_k)$ and $\hat{v}(t_k)$ denote the filtered position and velocity estimates, respectively.

These error signals are used to compute empirical noise statistics, generate residual plots, and analyze the dependence of estimation error on the magnitude of the velocity.

2.3. Sampling and Time Representation

The first column of the dataset represents physical time in seconds, recorded as a floating-point quantity. Although the sampling interval $\Delta t_k = t_k - t_{k-1}$ is not strictly constant, all time values correspond to real elapsed time rather than discrete tick indices.

Because Δt_k varies slightly from sample to sample, the state transition matrix of the Kalman filter is updated at each step using the measured time interval. This ensures that the prediction step correctly reflects the actual temporal spacing of the data.

3. Methods

3.1. Overview of the Processing Pipeline

The estimation pipeline consists of three main stages applied sequentially to the measured data. First, the raw measured signals are smoothed using Savitzky–Golay filters to attenuate high-frequency noise. Second, a discrete-time Kalman filter is applied to estimate position and velocity using a model driven by measured acceleration as an external input. Finally, estimation errors and residuals are computed relative to the reference signals for offline evaluation.

All processing is performed offline, and no constraints are imposed. The filtered estimates, along with the original measured signals and reference signals, are written to a unified output dataset to facilitate consistent visualization and quantitative analysis.

3.2. Savitzky–Golay Smoothing

Savitzky–Golay (SG) filtering is used to reduce high-frequency noise while preserving low-order signal trends. In this work, a 5-point quadratic SG smoother is applied independently to the measured position, velocity, and acceleration signals.

For a scalar signal x_k , the smoothed value $\tilde{x}_k$ is computed as a weighted sum of samples within a local window:

$$\tilde{x}_k = \sum_{i=-2}^2 c_i x_{k+i}, \quad (6)$$

where the coefficients correspond to a quadratic least-squares fit over a window of five samples. The resulting convolution kernel is given by

$$\mathbf{c} = \frac{1}{35} [-3, 12, 17, 12, -3]. \quad (7)$$

During initial samples where a full window is unavailable, the filter output is set equal to the raw measurement. Because the analysis is performed offline, the noncausal nature of the symmetric SG window does not pose a limitation.

3.3. Kalman Filter Model with Acceleration as Input

The Kalman filter state is defined as

$$\mathbf{x}_k = \begin{bmatrix} p_k \\ v_k \end{bmatrix}, \quad (8)$$

where p_k and v_k denote position and velocity at tick t_k . Measured acceleration is incorporated as a known external input rather than as a state variable.

The discrete-time state transition model is

$$\mathbf{x}_{k+1} = \mathbf{F}_k \mathbf{x}_k + \mathbf{G}_k a_k + \mathbf{w}_k, \quad (9)$$

with

$$\mathbf{F}_k = \begin{bmatrix} 1 & \Delta t_k \\ 0 & 1 \end{bmatrix}, \quad \mathbf{G}_k = \begin{bmatrix} \frac{1}{2} \Delta t_k^2 \\ \Delta t_k \end{bmatrix}, \quad (10)$$

where a_k is the measured (optionally SG-smoothed) acceleration input and Δt_k is the tick interval between samples.

The measurement model assumes direct observation of position and velocity:

$$\mathbf{z}_k = \begin{bmatrix} mpos(t_k) \\ mvel(t_k) \end{bmatrix} = \mathbf{H} \mathbf{x}_k + \mathbf{v}_k, \quad \mathbf{H} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}. \quad (11)$$

3.4. Noise Statistics and Covariance Design (R and Q)

Measurement noise statistics were empirically estimated using the difference between measured and reference signals:

$$n_x(t_k) = mpos(t_k) - rpos(t_k), \quad (12)$$

$$n_v(t_k) = mvel(t_k) - rvel(t_k), \quad (13)$$

$$n_a(t_k) = maccel(t_k) - raccel(t_k). \quad (14)$$

A total of $N = 814$ valid samples were used to compute the sample means, standard deviations, and covariance matrix.

The empirical noise statistics are:

$$\begin{aligned} \mu_x &= 0.3757, & \sigma_x &= 1.3622, \\ \mu_v &= 5.1345, & \sigma_v &= 40.7757, \\ \mu_a &= -31.4195, & \sigma_a &= 993.6126. \end{aligned}$$

The corresponding sample covariance matrix is

$$\mathbf{R}_{3 \times 3} = \begin{bmatrix} 1.8556 & -9.8784 & -150.3845 \\ -9.8784 & 1662.6606 & -11591.9941 \\ -150.3845 & -11591.9941 & 987265.9388 \end{bmatrix}. \quad (15)$$

For the Kalman filter implementation, only the position–velocity measurement block is used:

$$\mathbf{R} = \begin{bmatrix} 1.8556 & -9.8784 \\ -9.8784 & 1662.6606 \end{bmatrix}. \quad (16)$$

Acceleration is treated as an external input, and its variance is used to construct the process noise:

$$\sigma_a^2 = 987265.9388. \quad (17)$$

Although the Kalman filter assumes zero-mean Gaussian noise, the empirical means above are not exactly zero. This deviation is likely due to finite sample size and possible systematic offsets in the dataset rather than a fundamental violation of the Gaussian noise assumption. In this work, bias removal was not applied, and the empirical covariance values were used directly for filter tuning.

3.5. Kalman Filter Implementation

At each sample, the Kalman filter performs a standard predict–update cycle. During prediction, the state and covariance are propagated using the current tick interval Δt_k and acceleration input a_k . During update, the predicted state is corrected using the measured position and velocity.

Filter initialization is performed using the first available measured position and velocity samples. All computations are carried out using dynamically updated transition and noise matrices to accommodate variable tick intervals. The filtered position and velocity estimates, along with the acceleration input actually used by the filter, are recorded for subsequent analysis.

4. Results

After Kalman filtering, the estimated position and velocity signals are written to a separate output CSV file. These results are subsequently visualized and analyzed using MATLAB. The output dataset contains the original measured signals, the filtered estimates, and the reference signals, enabling direct time-domain comparison.

Results for acceleration are intentionally excluded from the analysis. In the proposed formulation, measured acceleration is treated as an external input to the Kalman filter, while the filter state consists only of position and velocity. As a result, the filter does not produce an independent acceleration estimate; discussing acceleration performance would therefore not provide additional insight into the estimator behavior. The results section focuses exclusively on position and velocity, which are the quantities directly estimated by the Kalman filter.

The following figures illustrate representative time-domain comparisons and error characteristics for the filtered position and velocity signals.

4.1. Time-Domain Position Comparison

Figure 1 shows the time-domain comparison between measured position, Kalman-filtered position, and the reference position. The measured position closely follows the reference trajectory but contains noticeable high-frequency noise. The Kalman-filtered position estimate exhibits significantly smoother behavior while remaining well aligned with the reference signal throughout the motion.

Minor deviations between the filtered estimate and the reference are observed near regions of high curvature in the trajectory. These deviations reflect the influence of the dynamic model and the integration of acceleration input, but they do not significantly affect overall tracking accuracy.

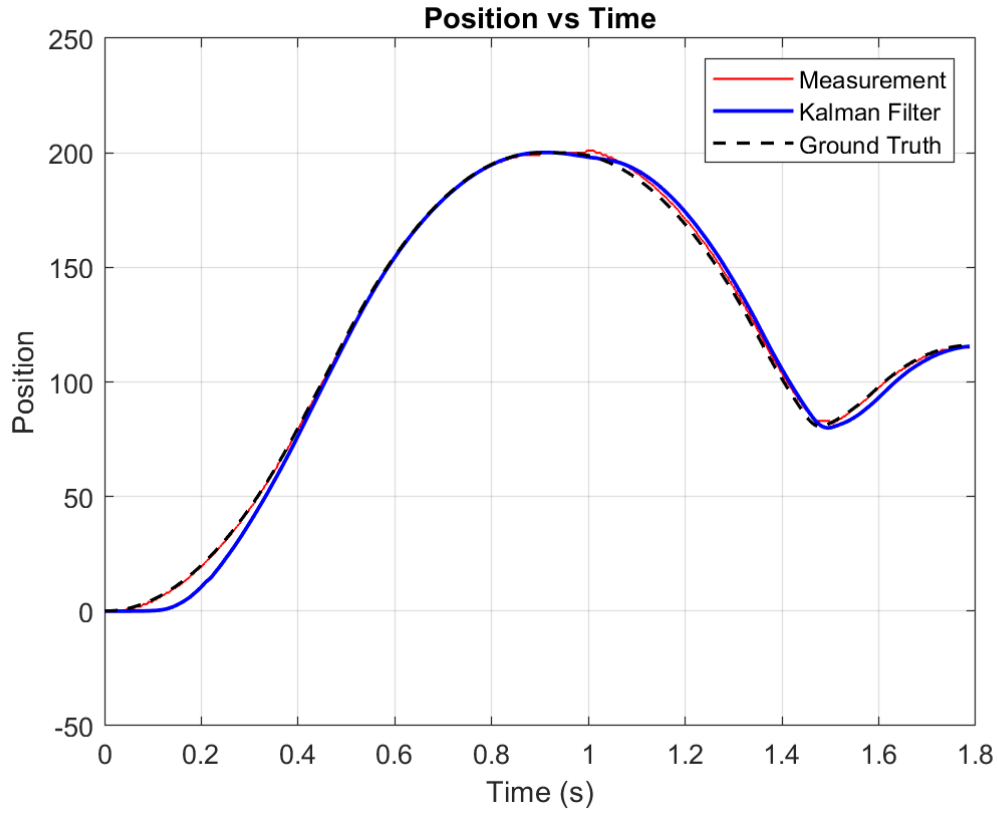


Figure 1: Position versus time for measured data, Kalman-filtered estimate, and reference signal.

4.2. Time-Domain Velocity Comparison

Figure 2 presents the velocity and time relationship. The measured velocity signal exhibits substantial noise and staircase-like behavior, particularly during low-speed intervals and near velocity reversals.

The Kalman-filtered velocity estimate substantially reduces high-frequency noise while preserving the overall shape and magnitude of the reference velocity. During rapid transitions, the filtered velocity exhibits slight smoothing and lag relative to the reference, reflecting the trade-off between noise suppression and responsiveness inherent in the filter tuning.

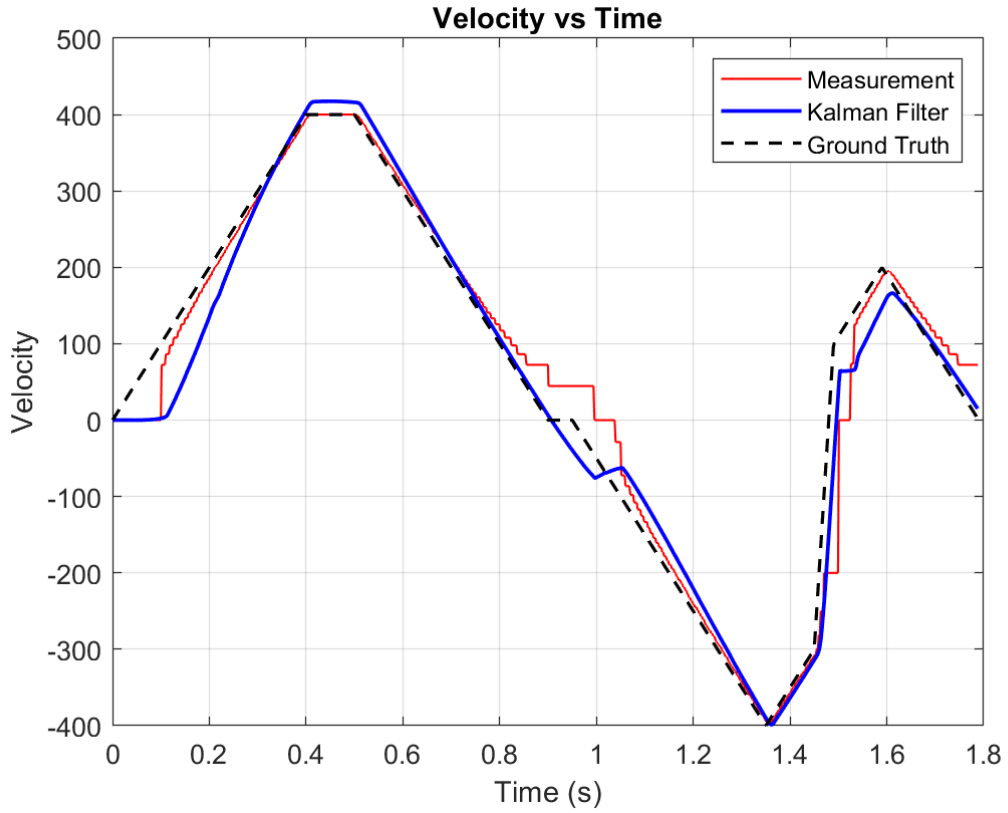


Figure 2: Velocity versus time for measured data, Kalman-filtered estimate, and reference signal.

4.3. Position Error Analysis

Figure 3 shows the position error relative to the reference signal for both the measured and Kalman-filtered position. The measured position error remains bounded but exhibits persistent high-frequency fluctuations.

The Kalman-filtered position error is smoother and more structured. Larger error magnitudes are observed during periods of sustained acceleration and deceleration, where small discrepancies in acceleration integration accumulate into position error before being corrected by measurement updates. Despite these effects, the filtered position error remains well behaved and free of high-frequency jitter.

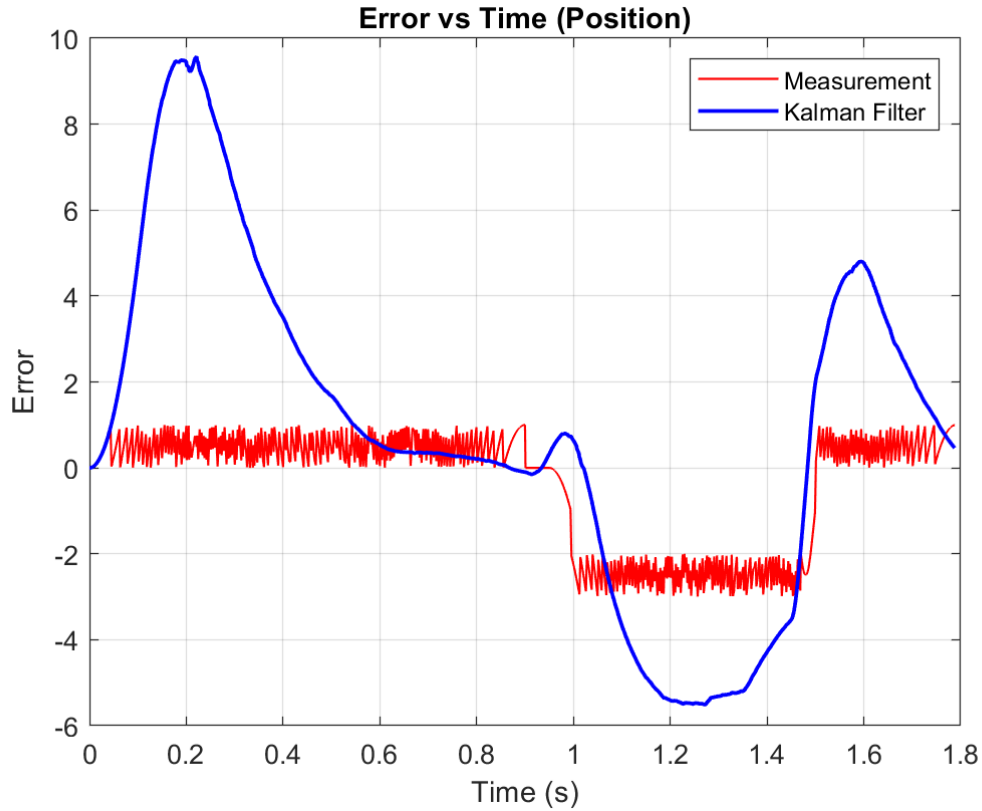


Figure 3: Position error versus time for measured data and Kalman-filtered estimate.

4.4. Velocity Error Analysis

Figure 4 illustrates the velocity error relative to the reference signal. The measured velocity error exhibits large spikes and high variance, particularly near velocity reversals and low-speed regimes. These spikes dominate the error behavior and indicate poor signal-to-noise ratio in the raw velocity measurement.

In contrast, the Kalman-filtered velocity error shows substantially reduced variance and fewer extreme outliers. Residual error remains during abrupt changes in velocity, reflecting the filter's reliance on a smoothed acceleration input and conservative process noise tuning. Overall, the filtered velocity error demonstrates a clear improvement over the raw measurement.

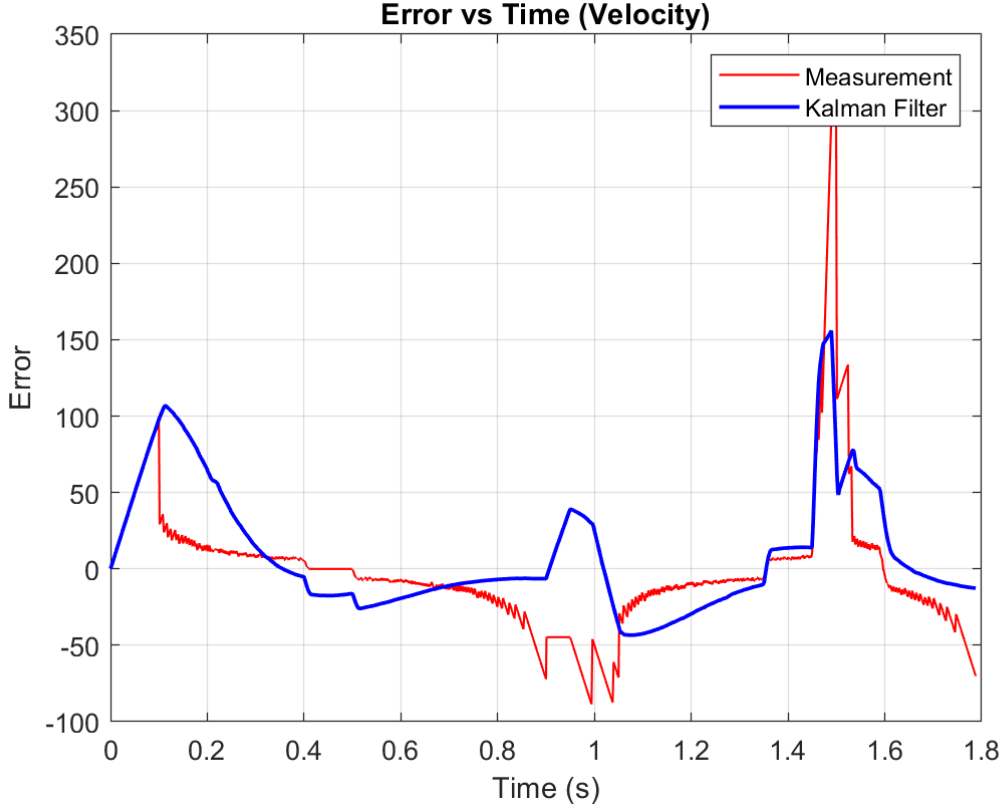


Figure 4: Velocity error versus time for measured data and Kalman-filtered estimate.

5. Discussion

The results demonstrate that the proposed Savitzky–Golay and Kalman filtering pipeline substantially improves encoder-derived velocity estimates. Beyond the final performance, the development process also provides insight into the impact of motion model selection on estimator behavior.

5.1. Limitations of the Constant-Acceleration State Model

An initial formulation of the Kalman filter employed a three-state constant-acceleration model with position, velocity, and acceleration included in the state vector. This model implicitly assumes that acceleration evolves smoothly over time, typically modeled as a constant or slowly varying quantity.

In practice, the measured acceleration signal exhibited abrupt changes that violated this assumption. Under these conditions, the Kalman filter was unable to adapt quickly enough, resulting in degraded tracking performance and increased transient error. Increasing the process noise associated with the acceleration state improved responsiveness but led to excessive noise in the estimated states, highlighting a fundamental trade-off imposed by the model structure.

5.2. Motivation for Using Acceleration as an Input

To address the limitations of the constant-acceleration state model, the final estimator formulation treats measured acceleration as an external input rather than as an estimated state. This change removes the requirement that acceleration evolve smoothly according to a predefined model.

By incorporating acceleration directly into the state transition, the Kalman filter leverages acceleration information to improve transient tracking while avoiding the need to estimate acceleration itself. This formulation proved more robust to abrupt changes in the measured acceleration signal and resulted in improved velocity estimates, as demonstrated in the Results section.

5.3. Primary Benefit in Velocity Estimation

Although both position and velocity are estimated by the Kalman filter, the primary benefit of the proposed approach lies in improved velocity estimation. The measured position signal is already relatively clean and closely aligned with the reference trajectory, limiting the magnitude of possible improvement.

In contrast, the measured velocity signal exhibits substantial noise amplification due to differentiation and encoder quantization effects. The Kalman filter significantly reduces this variance while preserving transient behavior, making velocity estimation the principal contribution of this work. The improvement in position estimation, while present, is comparatively modest.

5.4. Trade-offs and Remaining Limitations

Although the input-driven formulation improves robustness to rapid acceleration changes, it still relies on fixed noise tuning parameters. As a result, the estimator exhibits residual lag during highly aggressive motion, reflecting the balance between noise suppression and responsiveness. These limitations suggest that no single fixed model or tuning can optimally handle all motion regimes.

Nevertheless, the final design represents a practical and effective compromise for offline analysis of encoder-derived motion signals, achieving substantial noise reduction while maintaining acceptable responsiveness across a wide range of operating conditions.

5.5. Alternative Savitzky–Golay Derivative Formulation

In the present implementation, Savitzky–Golay smoothing was applied independently to position, velocity, and acceleration measurements. While this approach reduces high-frequency noise in each signal, it does not enforce consistency between derivatives.

A more principled alternative would be to apply Savitzky–Golay filtering exclusively to the position signal and compute velocity and acceleration directly from the fitted local polynomial. Because the Savitzky–Golay method is derived from a least-squares polynomial fit, analytical first- and second-order derivatives can be obtained from the same polynomial coefficients. This approach would produce internally consistent position, velocity, and acceleration estimates while reducing differentiation-induced noise amplification.

Exploring this formulation represents a promising direction for improving derivative quality and estimator consistency.

6. Next Steps

Several extensions would strengthen both the validity and applicability of this work:

1. **Acquire higher-quality and larger-volume datasets.** A standard Kalman filtering assumption is that process and measurement noise are well-modeled as zero-mean Gaussian. However, empirical noise estimates computed from the current dataset (even after smoothing) suggest deviations from this assumption. One plausible explanation is limited sample size, which can lead to biased or unstable estimates of means and covariances. A practical next step is to collect longer datasets (e.g., via FPGA logging) to improve the reliability of empirical noise characterization and filter tuning.

2. **Improve the noise modeling via propagation analysis.** The current approach estimates noise statistics directly from measured–reference differences. A complementary direction is to analytically propagate position noise into velocity and acceleration noise based on the discrete differentiation operations used to derive *mvel* and *maccel* in the data pipeline. This would provide a physics- and signal-processing-grounded model for $\text{Cov}([n_x, n_v])$ and $\text{Var}(n_a)$, and could help explain correlations and scaling effects across motion regimes.[1]
3. **Validate on dVRK data across representative operating ranges.** To assess external validity, the estimator should be tested on data spanning the typical range of dVRK motion (including low-speed motion, high-speed motion, and rapid transitions). This evaluation would confirm whether the observed trends generalize beyond the current dataset and would identify any regime-dependent tuning requirements.

A. Savitzky–Golay Coefficient Derivation

The Savitzky–Golay (SG) smoothing filter is derived by fitting a low-order polynomial to a local window of samples using least squares and evaluating the fitted polynomial at the center of the window. In this work, a quadratic polynomial is fit over a symmetric five-sample window.

Let the local sample index be $i \in \{-2, -1, 0, 1, 2\}$, and model the signal locally as

$$x(i) \approx a_0 + a_1 i + a_2 i^2. \quad (18)$$

Stacking the samples into a vector $\mathbf{y} = [x_{k-2}, x_{k-1}, x_k, x_{k+1}, x_{k+2}]^T$, the least-squares problem can be written as

$$\min_{\mathbf{a}} \|\mathbf{A}\mathbf{a} - \mathbf{y}\|^2, \quad (19)$$

where

$$\mathbf{A} = \begin{bmatrix} 1 & -2 & 4 \\ 1 & -1 & 1 \\ 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 2 & 4 \end{bmatrix}. \quad (20)$$

The solution is given by

$$\hat{\mathbf{a}} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{y}. \quad (21)$$

The smoothed signal value corresponds to the fitted polynomial evaluated at the center of the window, i.e.,

$$\tilde{x}_k = \hat{a}_0. \quad (22)$$

This operation can be expressed as a fixed linear combination of the data samples,

$$\tilde{x}_k = \sum_{i=-2}^2 c_i x_{k+i}, \quad (23)$$

with coefficients

$$\mathbf{c} = \frac{1}{35}[-3, 12, 17, 12, -3]. \quad (24)$$

For a fixed window size and polynomial degree, these coefficients are constant, and only the data within the sliding window changes at each step.

B. Kalman Filter Equations

This appendix summarizes the Kalman filter equations used in this work. The filter state is defined as

$$\mathbf{x}_k = \begin{bmatrix} p_k \\ v_k \end{bmatrix}, \quad (25)$$

where p_k and v_k denote position and velocity at time t_k .

Measured acceleration is treated as an external input a_k , and the discrete-time state transition model is

$$\mathbf{x}_{k+1} = \mathbf{F}_k \mathbf{x}_k + \mathbf{G}_k a_k + \mathbf{w}_k, \quad (26)$$

with

$$\mathbf{F}_k = \begin{bmatrix} 1 & \Delta t_k \\ 0 & 1 \end{bmatrix}, \quad \mathbf{G}_k = \begin{bmatrix} \frac{1}{2} \Delta t_k^2 \\ \Delta t_k \end{bmatrix}. \quad (27)$$

The measurement model assumes direct observation of position and velocity:

$$\mathbf{z}_k = \begin{bmatrix} mpos(t_k) \\ mvel(t_k) \end{bmatrix} = \mathbf{H} \mathbf{x}_k + \mathbf{v}_k, \quad \mathbf{H} = \mathbf{I}_2. \quad (28)$$

Process noise is modeled as uncertainty in the acceleration input, yielding a covariance

$$\mathbf{Q}_k = \mathbf{G}_k \sigma_{a,\text{eff}}^2 \mathbf{G}_k^T, \quad (29)$$

where $\sigma_{a,\text{eff}}^2 = \alpha \sigma_a^2$ is the effective acceleration noise variance. Measurement noise is modeled using the empirically estimated covariance matrix

$$\mathbf{R} = \text{Cov}([n_x, n_v]). \quad (30)$$

The Kalman filter recursion consists of the following steps:

Prediction:

$$\hat{\mathbf{x}}_{k|k-1} = \mathbf{F}_k \hat{\mathbf{x}}_{k-1|k-1} + \mathbf{G}_k a_k, \quad (31)$$

$$\mathbf{P}_{k|k-1} = \mathbf{F}_k \mathbf{P}_{k-1|k-1} \mathbf{F}_k^T + \mathbf{Q}_k. \quad (32)$$

Update:

$$\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}^T (\mathbf{H} \mathbf{P}_{k|k-1} \mathbf{H}^T + \mathbf{R})^{-1}, \quad (33)$$

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k (\mathbf{z}_k - \mathbf{H} \hat{\mathbf{x}}_{k|k-1}), \quad (34)$$

$$\mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{K}_k \mathbf{H}) \mathbf{P}_{k|k-1}. \quad (35)$$

These equations are evaluated at each time step using the corresponding interval Δt_k .

References

- [1] J. Y. Wu, Z. Chen, A. Deguet, and P. Kazanzides, “Fpga-based velocity estimation for control of robots with low-resolution encoders,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2018, pp. 1–8.